

Newsly

Project Report

Internship: App Development Internship

Task: Task 3

Application Name: Newsly

Developer: Nora Nagiy Sayed Hafez

Technology: Flutter

API: DummyJSON / REST API

HTTP Client: Dio

Local Caching: SharedPreferences

State Management: BLoC / Cubit

1. Project Overview

Newsly is a cross-platform Flutter application that fetches remote content from a REST API and displays it in a responsive news-style interface.

The application retrieves JSON data asynchronously, converts the response into structured Dart models, manages application state using Cubit, and presents the content through a scrollable user interface.

The application also supports search, categories, pull-to-refresh, detailed article views, and offline access to previously cached content.

2. Objectives

The main objectives of the project were to:

- Build a cross-platform Flutter application.
- Connect the application to a remote REST API.
- Fetch and process JSON data asynchronously.
- Implement clean state management using Cubit.
- Display remote content in a scrollable layout.
- Handle loading and error states.
- Implement retry functionality.
- Support offline access through local caching.
- Provide search and category filtering.
- Create a responsive and user-friendly interface.

3. Technologies Used

Flutter

Used to develop the cross-platform mobile application and user interface.

Dart

Used as the main programming language.

Dio

Used as the HTTP client for making asynchronous API requests.

DummyJSON

Used as the remote REST API that provides JSON content for the application.

BLoC / Cubit

Used for application state management and controlling the flow between the data layer and the user interface.

SharedPreferences

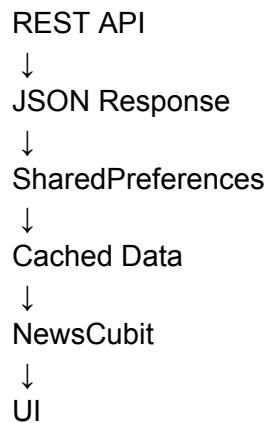
Used to cache the latest successfully retrieved news data locally for offline access.

4. Application Architecture

The application follows a layered structure that separates networking, data processing, state management, and presentation.

```
UI
↓
NewsCubit
↓
NewsRepository
↓
ApiService
↓
REST API
```

For offline access:



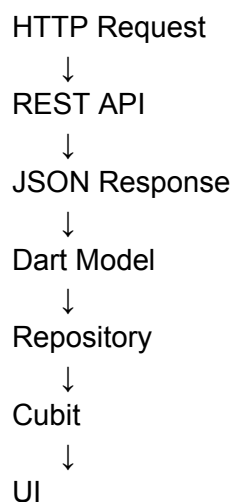
This structure makes the application easier to maintain and extend.

5. API Integration

The application connects to the DummyJSON REST API using Dio.

The API request is performed asynchronously and the returned JSON response is processed by the application.

The general request flow is:



The application extracts information such as:

- ID
- Title
- Body

- Tags
 - Views
 - Likes
 - Dislikes
-

6. JSON Parsing

The remote JSON response is converted into a Dart model using a factory constructor.

The `NewsModel` represents the application data and provides structured access to the API response.

This approach keeps JSON parsing separate from the user interface.

7. State Management

Cubit is used to manage the application's network and UI states.

The main states include:

```
NewsInitial
  ↓
NewsLoading
  ↓
NewsSuccess
```

If an operation fails:

```
NewsLoading
  ↓
NewsError
```

The `NewsSuccess` state also contains information about whether the application is displaying cached offline content.

8. Loading State

While the application is waiting for the API response, a loading indicator is displayed.

This provides visual feedback to the user and prevents the interface from appearing unresponsive during network operations.

9. Error Handling

The application handles failed network requests through an error state.

When an error occurs, the user is shown an appropriate message along with a **Try Again** button.

The retry button triggers another API request without requiring the user to restart the application.

10. Offline Support

The application stores the latest successfully retrieved news data locally using `SharedPreferences`.

When a network request fails and cached data is available, the application can display the previously retrieved content.

An offline indicator informs the user that cached content is currently being displayed.

Example:

No Internet Connection



Cached News Available



Display Cached Content

This improves the user experience when the device temporarily loses network connectivity.

11. Search Functionality

The application provides a search field that allows users to search through the available content.

The search functionality checks:

- News title

- News body
- News tags

The displayed list is updated dynamically as the user enters a search query.

12. Categories

The application automatically extracts categories from the tags returned by the API.

Users can select a category to filter the displayed news.

The available categories are presented horizontally using selectable chips.

The default category is:

All

13. Pull to Refresh

Users can refresh the news feed by pulling down on the screen.

The application sends a new request to the remote API and updates the displayed content with the latest response.

A refresh button is also available in the AppBar.

14. News Details

Users can tap any news card to open a dedicated details screen.

The details screen displays:

- Full title
- Full article content
- Views
- Likes
- Dislikes
- News tags

This provides a complete reading experience instead of displaying only a preview on the Home Screen.

15. User Interface

The application uses Flutter Material components to provide a clean and responsive interface.

The main interface includes:

- AppBar
- Search field
- Category chips
- News cards
- Refresh button
- Loading indicator
- Error screen
- Offline indicator
- News details screen

The layout is implemented using scrollable Flutter widgets to support different screen sizes.

16. Screenshots

Home Screen

The Home Screen displays the news feed retrieved from the remote API.

Search

The Search feature allows users to find specific content using keywords.

Categories

Users can filter news using the available categories.

News Details

Users can open a news item and view its complete details.

Offline Mode

Previously cached content can be displayed when the device is offline.

17. Testing

Test Case	Result
Fetch remote JSON data	Passd
Display news list	Passd
JSON parsing	Passd
Loading state	Passd
Error handling	Passd
Retry request	Passd
Search	Passd
Category filtering	Passd
Pull to refresh	Passd
News details	Passd
Local caching	Passd
Offline content	Passd
Responsive UI	Passd

18. Challenges

During development, several practical challenges were addressed, including:

- Working with asynchronous HTTP requests.
 - Parsing nested JSON responses.
 - Managing different application states using Cubit.
 - Handling failed network requests.
 - Implementing local caching.
 - Supporting offline content.
 - Combining search and category filtering.
 - Keeping the UI responsive during network operations.
-

19. Future Improvements

Possible future improvements include:

- Adding real news API sources.
 - Adding images for news articles.
 - Adding pagination or infinite scrolling.
 - Adding user authentication.
 - Adding bookmarks and saved articles.
 - Adding push notifications.
 - Adding more advanced offline synchronization.
 - Adding a favorites section.
-

20. Project Repository

The complete source code is available on GitHub:

GitHub Repository:

<https://github.com/nouranagiy/newsly-flutter>

The repository contains the Flutter source code, project structure, screenshots, and documentation.

21. Conclusion

Newsly demonstrates a complete Flutter application that communicates with a remote REST API and asynchronously processes JSON data.

The project demonstrates practical experience with HTTP networking, JSON parsing, Cubit state management, local caching, offline handling, search, filtering, refresh functionality, and responsive UI development.

The application provides a solid foundation for building larger cloud-connected mobile applications.